

Practica 2: Escalado de Sistemas **Distribuidos en la Nube**

Index

Introducción y Arquitectura Híbrida del Sistema	3
Componentes de la Arquitectura	3
Fase 1 y 2: Despliegue de la Infraestructura Base en AWS	5
Configuración de Red y Seguridad (Security Group)	5
Preparación de la Instancia EC2	5
2.3. Contenedorización de Servicios: Docker, RabbitMQ y Redis	5
Verificación de Conectividad Externa	6
Fase 3 (Ejercicios 1 y 2): Implementación del Auto-escalado Dinámico	7
Lógica del Trabajador Efímero (Ejercicio 1)	7
Orquestador y Primitiva de Stream (Ejercicio 2)	7
Pruebas de Carga y Demostración de Elasticidad	8
Fase 4 (Ejercicio 3): Procesamiento MapReduce con Lithops y Amazon S3	10
Preparación del Conjunto de Datos (Dataset)	10
Implementación del Patrón MapReduce	10
Estrategia de Despliegue Seguro en la Nube	11
Fase 4 (Ejercicio 4): Optimización mediante Procesamiento por Lotes (Batch)	13
El Algoritmo de Distribución de Carga	13
Refactorización del Flujo MapReduce	13
Validación y Resultados	13

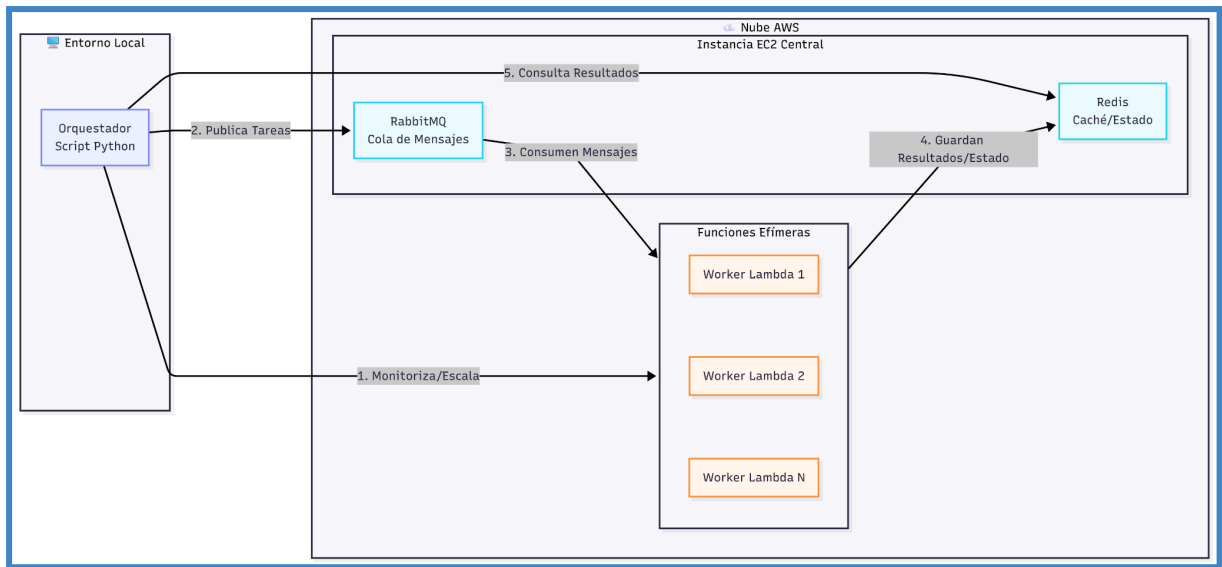
Introducción y Arquitectura Híbrida del Sistema

Este trabajo detalla el diseño y la implementación de un sistema distribuido escalable en la nube de AWS, correspondiente a la resolución de la segunda práctica de la asignatura. El objetivo principal de este proyecto es migrar el servicio de filtrado de insultos (InsultFilter) a la nube, demostrando de forma práctica la elasticidad del sistema. Esta elasticidad requiere que la infraestructura sea capaz de crear y destruir "trabajadores" automáticamente en función de la cantidad de texto o carga de trabajo que se necesite procesar en cada momento.

Componentes de la Arquitectura

La solución diseñada se estructura en tres bloques principales que interactúan para lograr el auto-escalado dinámico:

- **Nodo Central Fijo (Instancia AWS EC2):** El núcleo de almacenamiento y mensajería del sistema reside en una Máquina Virtual fija (EC2) desplegada en la nube. Para garantizar un entorno de ejecución estandarizado y libre de conflictos de dependencias, se ha utilizado Docker en esta máquina para contenerizar y ejecutar dos servicios fundamentales, un servidor RabbitMQ que gestiona la cola de mensajes a procesar, y una base de datos en memoria Redis encargada de almacenar los resultados finales.
- **Trabajadores Efímeros y Elásticos (AWS Lambda):** La capacidad de procesamiento pesado se ha delegado en el servicio AWS Lambda. Estas funciones actúan como trabajadores efímeros que se instancian únicamente cuando hay trabajo pendiente. Se encargan de recibir un lote de mensajes, aplicar el algoritmo de filtrado y censura de insultos, escribir los resultados atómicamente en la base de datos Redis de la EC2, y finalizar su ejecución para liberar recursos.
- **Orquestador de Auto-escalado (Monitorización Local):** El control de la elasticidad es gestionado por un script orquestador ejecutado fuera de las Lambdas. Este componente monitoriza de forma continua y pasiva el estado del servidor RabbitMQ remoto. Con base en el volumen exacto de mensajes encolados, el orquestador aplica una fórmula matemática para decidir el número óptimo de instancias Lambda que deben invocarse concurrentemente, asegurando que el sistema escale verticalmente de forma dinámica frente a picos de carga.



Imágen 1: Arquitectura-Relación Localhost + Cloud

Fase 1 y 2: Despliegue de la Infraestructura Base en AWS

Una vez definida la arquitectura híbrida, el primer paso consistió en establecer la arquitectura en la nube de Amazon Web Services (AWS). Para garantizar la comunicación fluida entre el entorno local (orquestador) y los trabajadores en la nube (Lambdas), fue necesario configurar exhaustivamente las reglas de red antes de instalar los servicios de mensajería y almacenamiento .

Configuración de Red y Seguridad (Security Group)

El diseño de red requirió la creación de un Grupo de Seguridad (Security Group) específico, denominado `practica2-sd`, actuando como un cortafuegos virtual. Para permitir el funcionamiento distribuido del sistema, se configuraron estrictamente las siguientes reglas de entrada (Inbound rules):

- **Puerto 22 (SSH):** Restringido a la IP local para la administración segura por terminal.
- **Puerto 5672 (Custom TCP):** Abierto de forma global (0.0.0.0/0) para permitir que las funciones AWS Lambda envíen y consuman eventos de la cola RabbitMQ .
- **Puerto 15672 (Custom TCP):** Habilitado para acceder al panel de control web de RabbitMQ desde el navegador local .
- **Puerto 6379 (Custom TCP):** Abierto (0.0.0.0/0) para que las Lambdas elásticas puedan persistir los resultados de los textos procesados en la base de datos Redis .

Preparación de la Instancia EC2

Con la red configurada, se procedió al despliegue de la máquina virtual. Se creó una instancia EC2 (denominada `Servidor-Mensajería`) utilizando una imagen de sistema operativo Ubuntu Server (versiones LTS compatibles con la capa gratuita) sobre una arquitectura t3.micro. Se vinculó el Grupo de Seguridad previamente creado y se generaron un par de claves criptográficas (.pem) para asegurar el acceso remoto mediante el protocolo SSH . Tras inicializar la instancia, se verificó el estado de ejecución y copiamos la Dirección IP Pública, para la interconexión de los componentes .

2.3. Contenedorización de Servicios: Docker, RabbitMQ y Redis

Para evitar conflictos de dependencias, garantizar la reproducibilidad del entorno y dar al sistema facilidad de configuración, se optó por la contenedorización de los servicios clave. Tras acceder a la EC2 por SSH, se instaló el motor de Docker. Posteriormente, se levantaron dos contenedores en segundo plano (--detach), mapeando sus puertos internos a los de la máquina anfitriona:

- **Redis:** Base de datos en memoria para el almacenamiento ultrarrápido de los resultados (puerto 6379) .
- **RabbitMQ:** Gestor de colas de mensajería (Broker). Se utilizó una imagen específica (rabbitmq:3-management) que incluye un panel web integrado para la monitorización visual del tráfico de la cola (puertos 5672 y 15672) .

Verificación de Conectividad Externa

La fase de infraestructura concluyó con una revisión de accesibilidad. Desde el equipo local, se accedió a través del navegador web a la dirección `http://[IP_PUBLICA_EC2]:15672`, confirmando que el entorno EC2 y el contenedor de RabbitMQ estaban respondiendo correctamente a peticiones externas y listos para recibir conexiones del orquestador y los workers en las siguientes fases de desarrollo.

En la siguiente imagen está un dashboard similar de lo que deberíamos haber visto, la imagen no coincide por con la arquitectura por falta de imágenes y el cierre de la cuenta de acceso para poder volver a testarlo.

Reglas de entrada

Reglas de salida

Compartiendo

Asociaciones de VPC

Recursos relacionados : *novedad*

Etiquetas

Reglas de salida (4)

Q Buscar

Administrar etiquetas

Editar reglas de salida

☐	Name	ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Interval...	Destino	Descripción
☐	-	sgr-0da30911cf274eee6	IPv4	SSH	TCP	22	85.87.6.111/32	-
☐	-	sgr-0813b0b5c8be18e6e	IPv4	TCP personalizado	TCP	5672	0.0.0.0/0	Para que las Lambdas envíen eventos a RabbitMQ
☐	-	sgr-0ca197bb61a1fa470	IPv4	TCP personalizado	TCP	6379	0.0.0.0/0	Para que las Lambdas guarden los resultados en Redis
☐	-	sgr-0c97c425aced64304	IPv4	TCP personalizado	TCP	15672	85.87.6.111/32	Para poder entrar al panel web de RabbitMQ desde mi navegador

Imagen 2: Security Groups de accesos y permisos dentro de la estructura

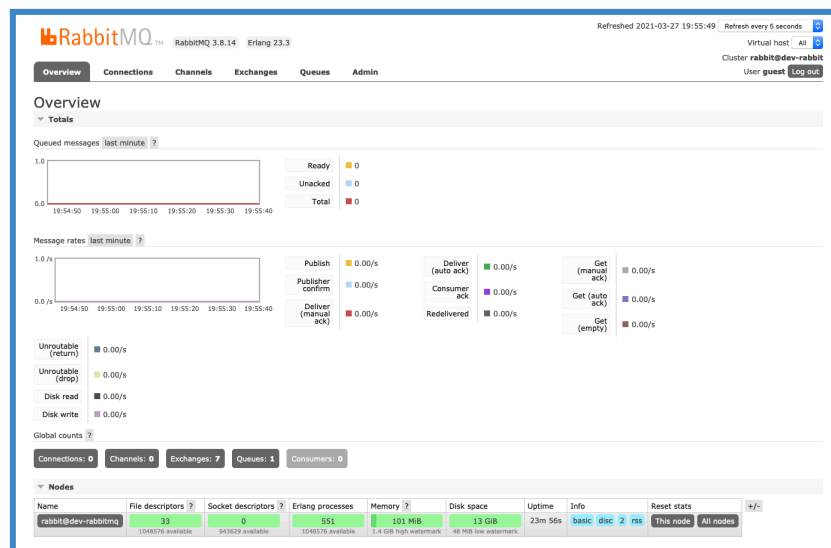


Imagen 3: Imagen similar al dashboard visible al rabbitMQ del EC2

Fase 3 (Ejercicios 1 y 2): Implementación del Auto-escalado Dinámico

El núcleo de la arquitectura distribuida desarrollada en esta práctica se centra en dotar al sistema de elasticidad, dando respuesta a los Ejercicios 1 y 2. El objetivo fundamental es abandonar el paradigma de trabajadores estáticos permanentemente encendidos y cambiar hacia un modelo dinámico. Para ello, se ha implementado un mecanismo capaz de lanzar instancias de funciones ("workers") bajo demanda, acotadas por un límite superior (`maxfunc`), escalando tanto hacia arriba (scale-up) frente a picos de trabajo, como hacia abajo (scale-down) cuando la cola de mensajería se vacía.

Para organizar el entorno y facilitar la comunicación remota sin necesidad de reconfigurar parámetros constantemente, las credenciales y la IP pública de la instancia EC2 se han centralizado en un archivo de configuración (`config.py`).

```
# Valor con la IP Pública actual de la instancia EC2 de AWS
AWS_EC2_IP = "13.216.244.177"

# Puertos estándar expuestos en los Security Groups
RABBITMQ_HOST = AWS_EC2_IP
RABBITMQ_PORT = 5672
QUEUE_NAME = "cola_insultos"

REDIS_HOST = AWS_EC2_IP
REDIS_PORT = 6379
```

Lógica del Trabajador Efímero (Ejercicio 1)

El procesamiento puro de los textos se ha encapsulado en el script `worker_lambda.py`, diseñado para emular la ejecución de una función AWS Lambda. Dada la naturaleza efímera de estas funciones (se inician, ejecutan su tarea y se destruyen), el trabajador recibe un lote de textos a través de un evento de entrada. El flujo de trabajo del worker es el siguiente:

- **Recepción y Filtrado:** Recibe un conjunto de mensajes, los itera y aplica el algoritmo de filtrado, censurando las palabras coincidentes con la lista negra de insultos.
- **Persistencia:** Para garantizar la consistencia en un entorno altamente concurrente y evitar condiciones de acceder al mismo valor, se conecta remotamente a la base de datos Redis de la instancia EC2 y utiliza Pipelines. Esto permite agrupar las operaciones de escritura (el incremento del contador total de insultos y la inserción del texto procesado en el histórico) ejecutándose de forma ordenada en la nube.

Orquestador y Primitiva de Stream (Ejercicio 2)

El control inteligente del clúster recae sobre el script `stream_auto_scaling.py`, ejecutado localmente, que implementa la primitiva requerida: `stream_operation(function,`

`maxfunc, queue_name`). Este orquestador monitoriza el estado de la infraestructura aplicando la siguiente lógica:

- **Monitorización Pasiva:** Se conecta remotamente a la cola RabbitMQ en AWS y evalúa el número exacto de mensajes pendientes. Esta consulta se realiza en modo pasivo (`passive=True`), obteniendo la métrica de carga sin alterar el estado de los mensajes.
- **Fórmula Matemática de Escalado:** Con el volumen de mensajes obtenido, se aplica una regla de aprovisionamiento, se asigna 1 worker por cada 20 mensajes encolados ($workers_teoricos = (num_mensajes // 20) + 1$). El número objetivo de trabajadores se calcula limitando este valor teórico mediante la variable `maxfunc` (para evitar sobrecostes o bloqueos por exceso de peticiones), resultando en `workers_objetivo = min(maxfunc, max(1, workers_teoricos))`.
- **Despliegue Dinámico:** Si los trabajadores actuales son insuficientes (`num_workers_actuales < workers_objetivo`), el sistema extrae lotes de hasta 20 mensajes de la cola y despliega nuevos hilos de ejecución concurrentes que invocan a la función empaquetada (Lambda), logrando así absorber la sobrecarga.

Pruebas de Carga y Demostración de Elasticidad

Para verificar el correcto funcionamiento del auto-escalado, se ha desarrollado un script generador de estrés (`inyectar_carga_aws.py`). Este componente publica una ráfaga masiva y repentina de 150 mensajes (con un mix de frases limpias y ofensivas) directamente en el broker remoto. Durante la ejecución concurrente, el comportamiento del sistema refleja la elasticidad esperada:

- El monitor detecta instantáneamente el pico de 150 mensajes.
- Se activa el evento de [Scale UP], instanciando al máximo de workers permitidos (ej. `maxfunc=5`) de forma simultánea.
- A medida que las funciones procesan sus respectivos lotes y la cola disminuye, el orquestador reduce progresivamente la creación de nuevas instancias, hasta estabilizar el sistema volviendo automáticamente a 0 trabajadores activos y 0 mensajes pendientes.

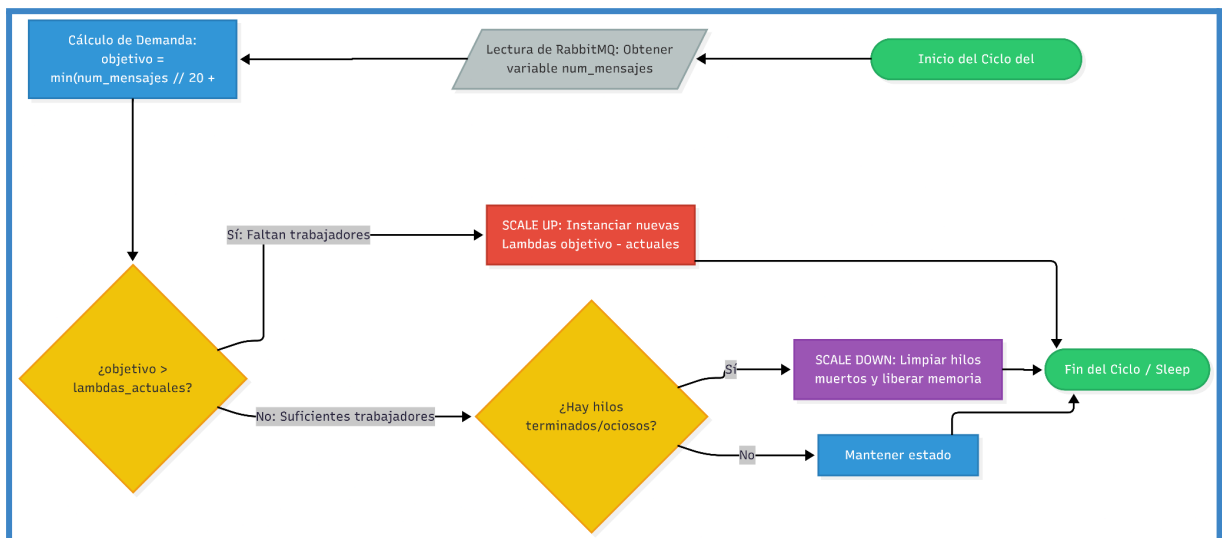
```
python .\stream_auto_scaling.py
Iniciizando monitor de auto-escalado para la cola: 'cola_insultos'
(Max Lambdas: 5)
[Monitor] Mensajes en Cola: 0 | Lambdas en ejecución: 0/5
[Monitor] Mensajes en Cola: 0 | Lambdas en ejecución: 0/5
[Monitor] Mensajes en Cola: 150 | Lambdas en ejecución: 0/5
[Escalado UP] Detectada sobrecarga. Lanzando 5 nueva(s) instancia(s)...
  [Lambda-feed301d] Instancia iniciada. Procesando lote de 20
mensajes...
  [Lambda-0dbf251e] Instancia iniciada. Procesando lote de 20
mensajes...
  [Lambda-30b0dfe1] Instancia iniciada. Procesando lote de 20
mensajes...
  [Lambda-06c6f436] Instancia iniciada. Procesando lote de 20
```



```

mensajes...
[Lambda-f33ad4d2] Instancia iniciada. Procesando lote de 20
mensajes...
[Lambda-feed301d] [+] Completada de forma elástica. Insultos
detectados: 10
[Lambda-0dbf251e] [+] Completada de forma elástica. Insultos
detectados: 16
[Lambda-30b0dfe1] [+] Completada de forma elástica. Insultos
detectados: 18
[Lambda-06c6f436] [+] Completada de forma elástica. Insultos
detectados: 24
[Lambda-f33ad4d2] [+] Completada de forma elástica. Insultos
detectados: 30
[Monitor] Mensajes en Cola: 50 | Lambdas en ejecución: 0/5
[Escalado UP] Detectada sobrecarga. Lanzando 3 nueva(s) instancia(s)...
[Lambda-c5ada236] Instancia iniciada. Procesando lote de 20
mensajes...
[Lambda-6356655b] Instancia iniciada. Procesando lote de 20
mensajes...
[Lambda-05d2521b] Instancia iniciada. Procesando lote de 10
mensajes...
[Lambda-05d2521b] [+] Completada de forma elástica. Insultos
detectados: 12
[Lambda-c5ada236] [+] Completada de forma elástica. Insultos
detectados: 18
[Lambda-6356655b] [+] Completada de forma elástica. Insultos
detectados: 24
[Monitor] Mensajes en Cola: 0 | Lambdas en ejecución: 0/5
[Monitor] Mensajes en Cola: 0 | Lambdas en ejecución: 0/5
[Monitor] Mensajes en Cola: 0 | Lambdas en ejecución: 0/5

```



Imágen 4: Diagrama de auto escalado de lambdas

Fase 4 (Ejercicio 3): Procesamiento MapReduce con Lithops y Amazon S3

El Ejercicio 3 de la práctica requiere la implementación del MapReduce para procesar masivamente una colección de archivos de texto. El objetivo es aplicar el filtro de censura de forma paralela sobre documentos almacenados en la nube (AWS S3) y obtener un conteo de los insultos encontrados. Para abstraer la complejidad del despliegue en la infraestructura y simplificar la ejecución del código en la nube, se ha utilizado el framework especializado Lithops.

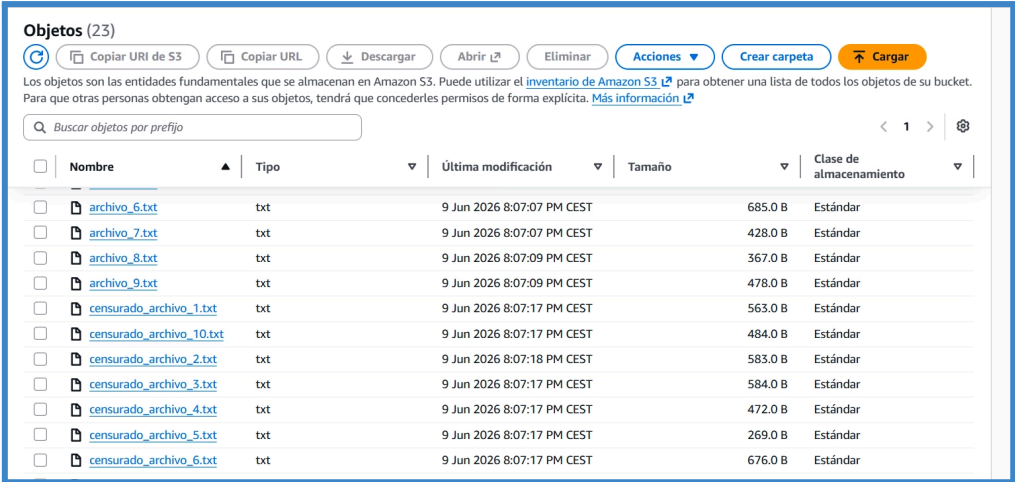
Preparación del Conjunto de Datos (Dataset)

Para alimentar el algoritmo, se implementó un script auxiliar (`generar_archivos_s3.py`) encargado de sintetizar un entorno de pruebas realista. Este generador crea automáticamente una colección de archivos .txt que combinan de forma probabilística líneas de texto completamente limpias con frases que contienen insultos de nuestra lista negra. Esta colección simula los textos en bruto que residirán en nuestro repositorio de almacenamiento en la nube.

Implementación del Patrón MapReduce

El núcleo de la solución radica en el script `map_reduce_lithops.py`, que divide el procesamiento en dos fases perfectamente diferenciadas y orquestadas por Lithops:

- **Fase MAP (Mapeo Paralelo):** Se ha definido la función `map_filter_insults(nombre_archivo, storage)`. El orquestador inyecta un archivo distinto a cada instancia Lambda ejecutada en paralelo. Cada función se encarga de utilizar el cliente de almacenamiento (storage) para descargar su archivo asignado desde el bucket de S3, censurar el contenido mediante el algoritmo de filtrado, y volver a subir un nuevo fichero limpio al mismo bucket utilizando el prefijo censurado_. Como valor de retorno, cada Lambda devuelve el número de insultos detectados en su documento específico.



Objetos (23)					
Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el inventario de Amazon S3 para obtener una lista de todos los objetos de su bucket. Para que otras personas obtengan acceso a sus objetos, tendrá que concederles permisos de forma explícita. Más información					
☐	Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
☐	archivo_6.txt	txt	9 Jun 2026 8:07:07 PM CEST	685.0 B	Estándar
☐	archivo_7.txt	txt	9 Jun 2026 8:07:07 PM CEST	428.0 B	Estándar
☐	archivo_8.txt	txt	9 Jun 2026 8:07:09 PM CEST	367.0 B	Estándar
☐	archivo_9.txt	txt	9 Jun 2026 8:07:09 PM CEST	478.0 B	Estándar
☐	censurado_archivo_1.txt	txt	9 Jun 2026 8:07:17 PM CEST	563.0 B	Estándar
☐	censurado_archivo_10.txt	txt	9 Jun 2026 8:07:17 PM CEST	484.0 B	Estándar
☐	censurado_archivo_2.txt	txt	9 Jun 2026 8:07:18 PM CEST	583.0 B	Estándar
☐	censurado_archivo_3.txt	txt	9 Jun 2026 8:07:17 PM CEST	584.0 B	Estándar
☐	censurado_archivo_4.txt	txt	9 Jun 2026 8:07:17 PM CEST	472.0 B	Estándar
☐	censurado_archivo_5.txt	txt	9 Jun 2026 8:07:17 PM CEST	269.0 B	Estándar
☐	censurado_archivo_6.txt	txt	9 Jun 2026 8:07:17 PM CEST	676.0 B	Estándar

Imágén 5: Creación de censuras de archivos en el Bucket S3

- **Fase REDUCE (Agregación de Resultados):** Una vez que todas las instancias de la fase MAP han finalizado su ejecución concurrente, Lithops invoca automáticamente la función `reduce_total_insults(resultados_mapas)`. Esta función recibe una lista con los conteos individuales emitidos por los mappers y ejecuta una suma agregada para devolver el informe global de insultos censurados en toda la colección.

Estrategia de Despliegue Seguro en la Nube

Para garantizar el éxito de la ejecución en AWS Academy, la transición a la nube se realizó mediante la configuración externa del entorno, manteniendo el código del script MapReduce prácticamente inalterado:

- **Prueba en Localhost:** Inicialmente, Lithops se configuró contra el backend localhost en un contenedor docker por su alta facilidad de configuración en bases linux y Python 3.10. El script sube los archivos de prueba a un almacenamiento virtual, valida la exactitud matemática del MapReduce y confirma que los documentos censurado_*.txt se generan correctamente.
- **Transición a AWS (Producción):** Tras la validación, se creó un archivo de configuración `.lithops_config`. En él, se establecieron como backends reales `aws_lambda` para el cómputo y `aws_s3` para el almacenamiento, inyectando las credenciales de sesión de AWS Academy (`aws_access_key_id`, `aws_secret_access_key`, `aws_session_token`) y el identificador único (ARN) del LabRole.

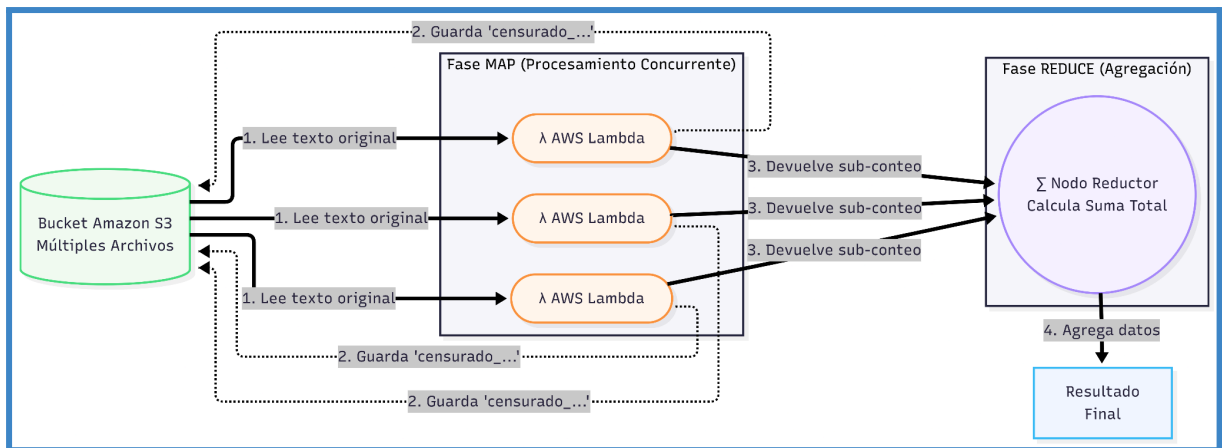
```
lithops:
    backend: aws_lambda
    storage: aws_s3

aws:
    access_key_id: <TU_ACCESS_KEY>
    secret_access_key: <TU_SECRET_KEY>
    session_token: <TU_SESSION_TOKEN>
    region_name: us-east-1

aws_lambda:
    execution_role: <PEGA_AQUÍ_EL_ARN_DEL_LABROLE>
    region_name: us-east-1

aws_s3:
    storage_bucket: datos-practica2-sd-ivan-pere
    region_name: us-east-1
```

- **Ejecución Final:** En la consola web de Amazon, se aprovisionó un bucket real con un nombre globalmente único. Al ejecutar el script configurado, Lithops subió los archivos originarios a AWS S3, orquestó las funciones Lambda reales y escribió los resultados en la nube en cuestión de milisegundos.



Imágen 6: Arquitectura y flujo del MapReduce

Fase 4 (Ejercicio 4): Optimización mediante Procesamiento por Lotes (Batch)

Como final de la práctica, el Ejercicio 4 aborda la eficiencia de costes y recursos. En la arquitectura clásica de MapReduce (implementada en el Ejercicio 3), el sistema aprovisiona una función Lambda por cada archivo procesado. Si bien esto proporciona un paralelismo absoluto, resulta ineficiente y genera sobrecostes cuando se trata de una colección masiva de archivos muy pequeños. Para solucionar esto, se ha implementado una operación Batch (procesamiento por lotes) utilizando la firma `operacion_batch(function, maxfunc, bucket)`. El objetivo es agrupar el trabajo, de forma que un número reducido de Lambdas procese múltiples archivos concurrentemente mediante bucles internos, demostrando así una gestión eficiente de los recursos bajo un límite estricto maxfunc.

El Algoritmo de Distribución de Carga

El núcleo de esta optimización reside en la transición de un archivo por Lambda a uno de un lote de archivos por Lambda. Para ello, el orquestador escanea el contenido del bucket S3 y recupera la lista total de archivos a procesar. A continuación, aplica un algoritmo de reparto equitativo a repartir en montones. Si el sistema detecta 10 archivos pero el límite de concurrencia está establecido en maxfunc = 3, la colección original no dispara 10 Lambdas. En su lugar, se divide en 3 sub-listas o chunks equilibrados. Como resultado, solo se instanciarán 3 funciones AWS Lambda simultáneas, reduciendo drásticamente el consumo computacional en la nube.

Refactorización del Flujo MapReduce

Para adaptar el procesamiento a esta nueva agrupación, el código del framework Lithops fue refactorizado, conservando la Fase REDUCE intacta pero alterando la Fase MAP:

- **Fase MAP Batch** (`map_filter_insults_batch`): A diferencia del ejercicio anterior, esta función ahora recibe como parámetro de entrada una lista completa de nombres de archivo. En su interior, ejecuta un bucle iterativo donde descarga cada archivo de su lote desde el S3, aplica la función de censura de insultos, sube la versión *censurado_* a la nube y acumula el número de palabras censuradas. Una vez procesado todo su lote, devuelve el subtotal de insultos encontrados.
- **Orquestación**: La invocación a Lithops `fexec.map_reduce()` se realiza pasando directamente la matriz de chunks a la función MAP optimizada, y conectando la salida con la función original `reduce_total_insults()` para obtener el conteo global.

Validación y Resultados

La verificación de esta arquitectura se realizó regenerando los 10 archivos de prueba en el entorno y lanzando el script de optimización con el límite maxfunc en un valor inferior al total de archivos. La ejecución certificó que la carga de trabajo en la nube se limitó estrictamente a la cantidad de workers configurada. Cada trabajador procesó entre 3 y 4 documentos de

forma ininterrumpida, finalizando con la correcta persistencia de todos los archivos filtrados en Amazon S3 y garantizando la coherencia en el total de censuras emitidas por la función Reduce.

```
root@02d31fd3c006:/app# python lithops_functions.py
[+] Inicializando ejecutor de Lithops...
2026-06-11 12:06:06,393 [INFO] config.py:139 -- Lithops v3.6.4 -
Python3.10
2026-06-11 12:06:06,627 [INFO] aws_s3.py:59 -- S3 client created -
Region: us-east-1
2026-06-11 12:06:07,471 [INFO] aws_lambda.py:97 -- AWS Lambda client
created - Region: us-east-1
[*] Subiendo archivos frescos al bucket real de AWS...
[..] Escaneando archivos dentro del bucket
'datos-practica2-sd-ivan-pere'...
[>] Disparando Operación Batch: 11 archivos repartidos en solo 3 Lambdas
concurrentes.
2026-06-11 12:06:10,311 [INFO] invokers.py:119 -- ExecutorID 27d997-0 |
JobID M000 - Selected Runtime: default-runtime-v310 - 256MB
2026-06-11 12:06:10,572 [INFO] invokers.py:186 -- ExecutorID 27d997-0 |
JobID M000 - Starting function invocation: map_filter_insults_batch() -
Total: 3 activations
2026-06-11 12:06:10,578 [INFO] invokers.py:225 -- ExecutorID 27d997-0 |
JobID M000 - View execution logs at
/tmp/lithops-root/logs/27d997-0-M000.log
2026-06-11 12:06:10,580 [INFO] wait.py:105 -- ExecutorID 27d997-0 -
Waiting for 20% of 3 function activations to complete
```

```
2026-06-11 12:06:15,608 [INFO] invokers.py:119 -- ExecutorID 27d997-0 |
JobID R000 - Selected Runtime: default-runtime-v310 - 256MB
2026-06-11 12:06:15,855 [INFO] invokers.py:186 -- ExecutorID 27d997-0 |
JobID R000 - Starting function invocation: reduce_total_insults() -
Total: 1 activations
2026-06-11 12:06:15,856 [INFO] invokers.py:225 -- ExecutorID 27d997-0 |
JobID R000 - View execution logs at
/tmp/lithops-root/logs/27d997-0-R000.log
2026-06-11 12:06:15,856 [INFO] executors.py:494 -- ExecutorID 27d997-0 -
Getting results from 1 function activations
2026-06-11 12:06:15,857 [INFO] wait.py:101 -- ExecutorID 27d997-0 -
Waiting for 3 function activations to complete
```

3/3

```
2026-06-11 12:06:23,883 [INFO] executors.py:618 -- ExecutorID 27d997-0 -
Cleaning temporary data
```

[+] [REPORTE BATCH FINALIZADO - EJERCICIO 4]

Total de insultos censurados: 29

```
root@02d31fd3c006:/app#
```

